

# OOP Project: Food Delivery System (Glovo, Bolt Food, UberEats)

By Rotea Dragoş (Group 30424)

## 1. Intentions with the application:

### App description:

- I want to build an app similar to Glovo, Wolt or Bolt Food. This app is supposed to allow my user to order meals from restaurants in his area.

### Must-do requirements:

- I should have three **users**: **admin**, **app\_user** (customer) and **driver**.
- The **admin** should be able to **delete** and **add** accounts of **drivers** and also **delete** and **add restaurants** on the app. Admins should also be able to **view** the **orders** from the **customers**.
- The **customers** should be able to **browse restaurants**. They should be able to **browse** the **menu** after they **decide** on a specific **restaurant** and **put** the desired **food options** in a **check-out** and finally **place** the **order**.
- The **drivers** should be able to **view** the **orders** assigned to them by the app. When the driver goes to different stages with their order (picked up, being delivered, arriving, and ordered delivered successfully), the **driver** should be able to **update** the **order status**.

### Nice to have:

- The **customer** will be able to **view** the **list of orders** made previously on his account (order history).

## 2. UML Diagram: (initial version)

A. We start with the requirements:

a. Identify what the system can do:

All users can register.

- **Admins/Managers** can add/delete restaurants, add/delete drivers and view orders.
- **Customers** can browse restaurants, place orders, view order history.
- **Drivers** can view assigned orders, update order status.

b. Use cases:

Register, Login, Add/Delete Restaurant, Add/Delete Drivers, View Orders, Browse Restaurants, Place Orders, View Order History, View Assigned Orders, Update Order Status

B. Identify actors (users and any other system that interacts with the app) and use cases:

a. Admins, Customers, Drivers

C. Extract classes:

a. Admin, Customer, Driver, Order, Restaurant, Menu, Menu Item

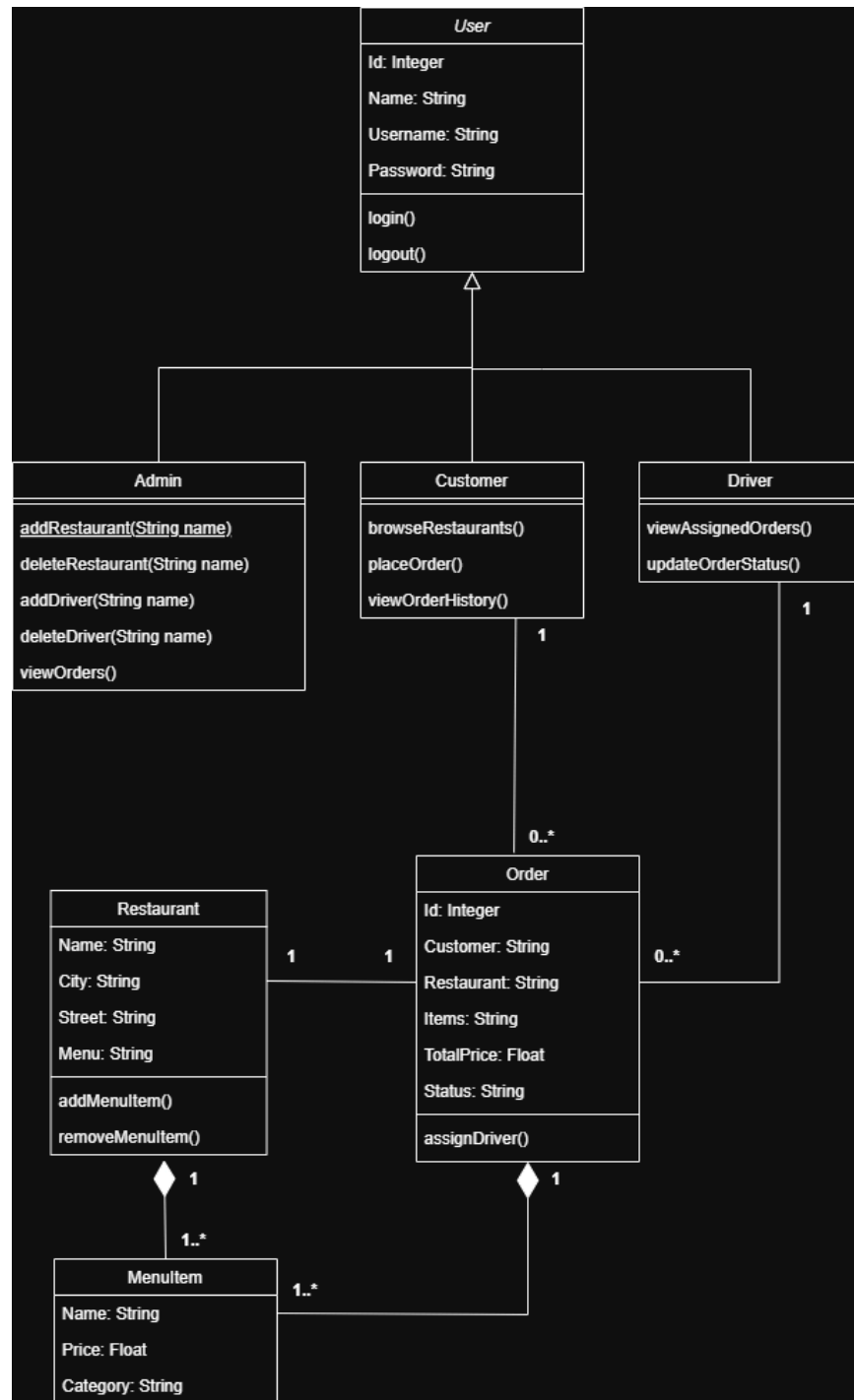
D. For each class extract attributes, methods:

- User: id, name, username, password, login(), logout()
- Admin: id, name, username, password, login(), logout(), addRestaurant(), deleteRestaurant(), addDriver(), deleteDriver(), viewOrders()
- Customer: id, name, username, password, login(), logout(), browseRestaurants(), placeOrder(), viewOrderHistory()
- Driver: id, name, username, password, login(), logout(), viewAssignedOrders(), updateOrderStatus()
- Restaurant: name, city, street, menu, addMenuItem(), removeMenuItem()
- MenuItem: name, price, category
- Order: id, customer, restaurant, items, totalPrice, status, assignDriver()

E. Identify relationships:

- Admin inherits User
- Customer inherits User
- Driver inherits User

- Restaurant & MenuItem → composition
- Restaurant & Order → association
- Order & MenuItem → association
- Order & OrderItem → composition
- OrderItem & MenuItem → association
- One Customer can have many Orders
- One Order can be from only one Restaurant
- One Order can have many MenuItems
- One Restaurant can have many MenuItems



### 3. UML Diagrams: (updated final version)

Furthermore, I will present the updated version of my UML with all the new classes and layers to it, explaining each layer and class step by step:

#### A. Architecture overview:

Each layer only communicates with the layer below it, following good OOP principles:

- ❖ **View Layer** → the user interface (swing)
- ❖ **Service Layer** → business logic (handles the communication between the database and the interface)
- ❖ **Database Layer** → supported by the data access object classes who extract the necessary data from the data base and use it in our app
- ❖ **Model Layer** → our main classes and objects that define the actors in our app (contain constructors, setters, getters and attributes)
  
- ❖ **Exceptions Layer** → custom error handling (throw-catch blocks)
- ❖ **Test Layer** → logic validation for some specific used methods

#### a) Model Layer:

- **User** → ABSTRACT class: base for all system users, has polymorphic behaviour for dashboards, and enforces role validation. User is the superclass of Customer, Driver, and Admin.
- **Customer, Driver, Admin** → EXTEND User, each with their attributes from user and their specific roles in the app
- **Restaurant** → a restaurant registered in the system with its attributes (used both by Customers to browse and Admins to manage)
- **MenuItem** → a food item belonging to a restaurant with its own attributes. Each MenuItem belongs to exactly one restaurant.
- **Order** → a customer order with specific attributes.

## b) Database Layer:

- **DatabaseConnection** → manages the JDBC (Java Database Connectivity), centralizes the DB connection and configuration
- **UserDAO** → handles all operations related to users like the login, registerUser, mapping the DB rows to the User subclasses
- **RestaurantDAO** → handles all restaurant related methods and functions, fetching all restaurants, checking for duplicates in the DB, saving new restaurants and deleting them by ID
- **MenuDAO** → handles menu functions like fetching all menu items of a restaurant (used when the user clicks on a restaurant to view the menu)
- **OrderDAO** → allows the customers to place orders and drivers to accept and complete them, updating their order status

## c) Service Layer: (bridge between DB and interface)

- **AuthService** → handles authentication and registration by validating input, login logic and throwing meaningful exceptions
- **RestaurantService** → contains business rules for restaurants, preventing empty fields and duplicate restaurant names
- **OrderService** → encapsulates order workflow logic, preventing empty orders and helping with placing orders and assigning/completing them to drivers.

## d) View Layer:

- **Main Frame** → main application window, using CardLayout for navigation and controlling transitions between dashboards.
- **LoginPanel & SignUpPanel** → handle authentication for both sign up and login user interface, delegating the logic to the AuthService
- **CustomerDashboard** → creates a dashboard where customers can browse restaurants, view menus, build their shopping cart and place orders.
- **DriverDashboard** → lets drivers view available orders, accept and complete orders.

- **AdminDashboard** → helps admins add and delete restaurants by id, having the role of an administrative control panel.

**e) Exception Layer:**

- **InvalidLoginException** → thrown when authentication fails.
- **InvalidRoleException** → thrown when an invalid role is used, preventing unauthorized access.

**f) Test Layer:**

- **OrderTest** → testing core logic outside of GUI, and verifying the total price calculation.
- **SecurityTest** → ensures role validation works correctly for both a valid and invalid role entry, confirming exceptions are thrown as expected.

## **B. Relationships in the UML:**

1. Inheritance:

- ★ Customer, Driver, Admin inherit from User

2. Association:

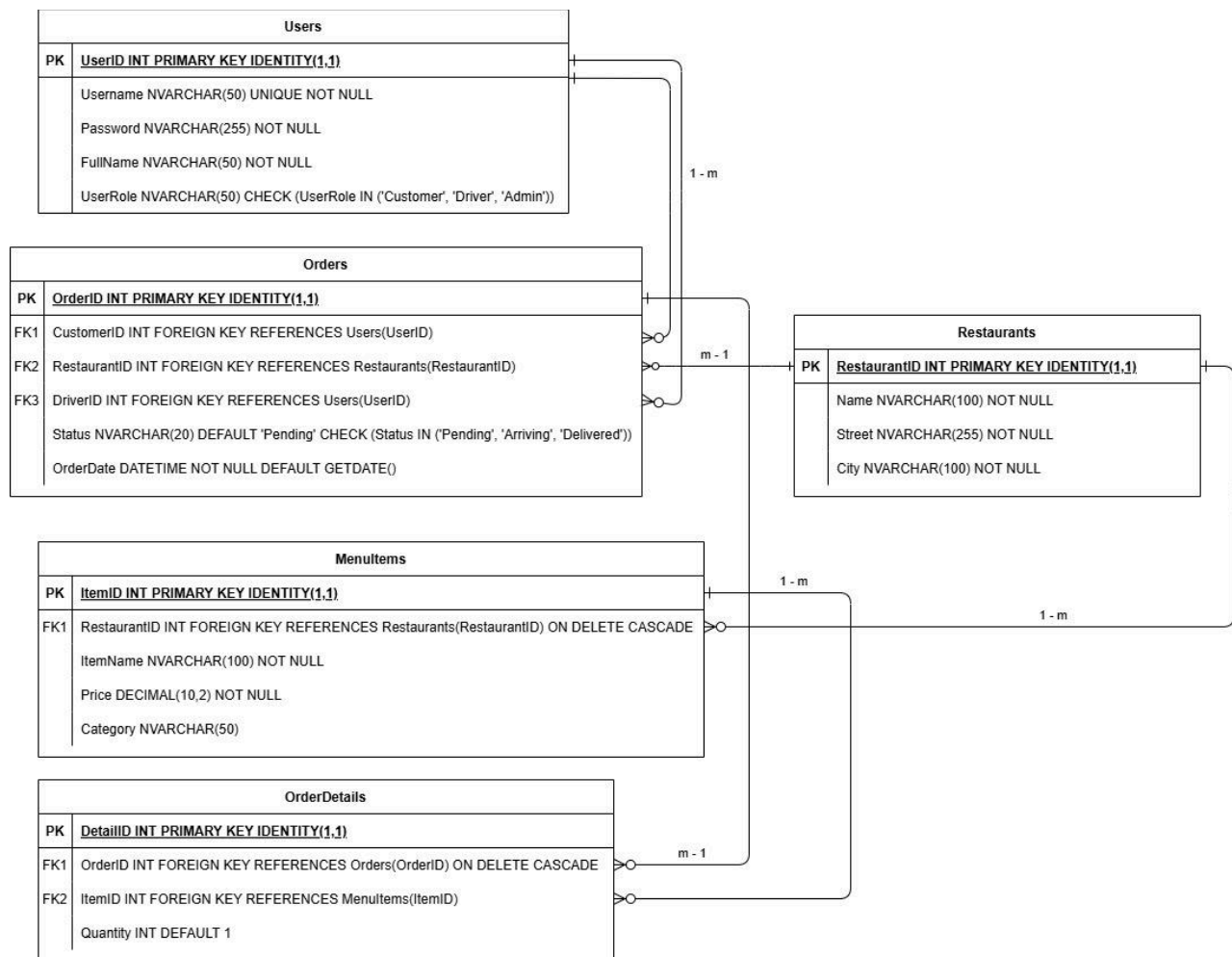
- ★ Customer - Order: a customer can place multiple orders
- ★ Driver - Order
- ★ Restaurant - Order

3. Composition:

- ★ Order - MenuItem - menu items cannot exist independently
- ★ Restaurant - MenuItem

#### 4. Dependency:

- ★ View - Service (LoginPanel - AuthService / CustomerDashboard - OrderService, RestaurantService / AdminDashboard - RestaurantService)
- ★ Service - Database (AuthService - UserDAO / OrderService - OrderDAO / RestaurantService - RestaurantDAO, MenuDAO)
- ★ Database - Model (UserDAO - User / OrderDAO - Order / RestaurantDAO - Restaurant)



## **4. Lifecycle of an order:**

### **1. Customer Authentication:**

- The user logs in through the LoginPanel
- Credentials are validated by AuthService
- A Customer object is returned upon successful authentication
- The system routes the user to the CustomerDashboard

### **2. Restaurant Selection:**

- The CustomerDashboard requests available restaurants from RestaurantService
- RestaurantService retrieves restaurant data via RestaurantDAO
- Restaurants are displayed in the UI for selection

### **3. Menu Loading:**

- When a restaurant is selected, the dashboard requests its menu
- RestaurantService fetches MenuItem objects associated with the restaurant
- Menu items are displayed dynamically

### **4. Shopping Cart Creation and Validation:**

- The customer adds MenuItem objects to the cart
- The system enforces the rule that all items must belong to the same restaurant
- The total price is calculated in real time

## **5. Order Creation:**

- Upon checkout, an Order object is created
- The order is associated with the current Customer and the selected Restaurant
- Selected MenuItem objects are added to the order
- The initial order status is set to PENDING

## **6. Order Persistence:**

- The Order is passed to OrderService
- Business validation is applied
- OrderService stores the order using OrderDAO
- A unique order ID is generated and assigned

## **7. Order Assignment:**

- Drivers retrieve available orders through DriverDashboard
- A Driver may accept an order
- The order status is updated to ARRIVING
- The order is now associated with the selected driver

## **8. Order Delivery**

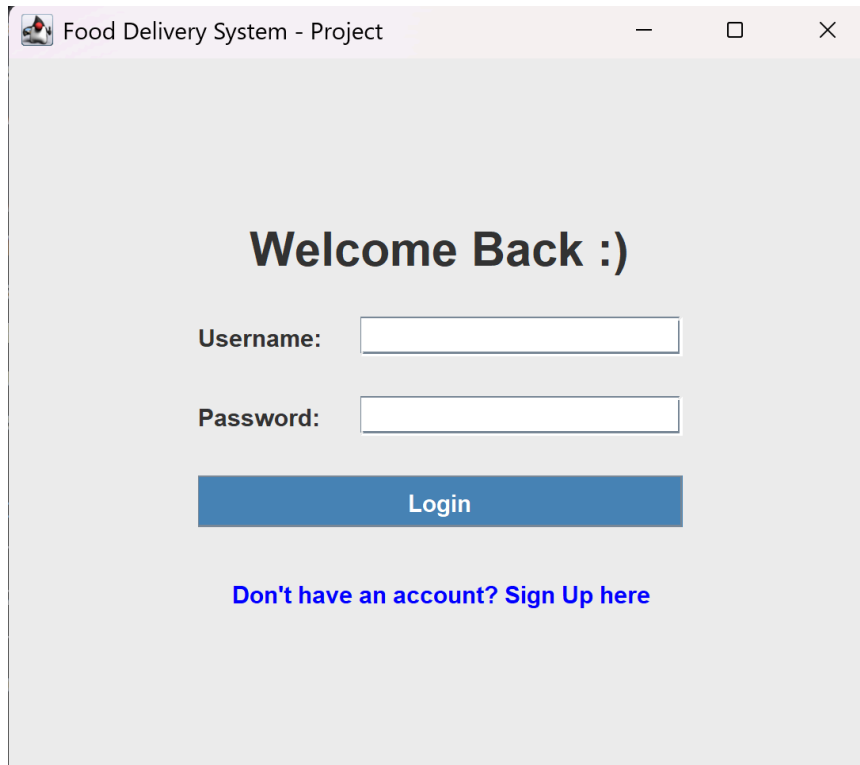
- The driver completes the delivery
- The order status is updated to DELIVERED
- The order lifecycle is completed

## **9. System Consistency:**

- Order status transitions are controlled using the OrderStatus enum

- All operations follow a layered architecture:
  - View → Service → DAO → Model
- Each layer has a single responsibility

## 5. Screenshots of the GUI:



 Create New Account

Full Name:

Username:

Password:

I am a:

Customer

▼

Back to Login

Create Account

 Administrator Panel - Andrei Ionescu

### Admin Panel: Register New Restaurant

Restaurant Name:

Street:

City:

Save Restaurant

Remove Restaurant

Enter Restaurant ID:

Delete by ID

Customer Portal - Welcome, Elena Dumitru

1. Select a Restaurant

Zama (Strada Napoca 16, Cluj-Napoca)

Marty Restaurants (Strada Victor Babeş 39, Cluj-Napoca)

Samsara Foodhouse (Strada Cardinal Iuliu Hossu 3, Cluj-Napoca)

Pizzeria Euphoria (Calea Turzii 7, Cluj-Napoca)

Cimbru Author Eatery (Strada Decebal 37, Cluj-Napoca)

Bujole (Piaţa Unirii 15, Cluj-Napoca)

Indigo Indian (Strada Piezişă 10, Cluj-Napoca)

MEAT UP (Strada Eroilor 14, Cluj-Napoca)

Garlic Bites (Strada Inocenţiu Micu Klein 11, Cluj-Napoca)

Tokyo Japanese (Strada Gheorghe Marinescu 5, Cluj-Napoca)

Express Chef (Strada Avram Iancu 492, Floreşti)

Pizza Mania (Strada Eroilor 15, Floreşti)

Burger Island (Strada Cetăţii 2, Floreşti)

Traditional Apahida (Strada Libertăţii 20, Apahida)

Wok'n'Roll (Iulius Mall Food Court, Cluj-Napoca)

Noodle Pack (Vivo! Mall, Cluj-Napoca)

Eggcetera (Strada Napoca 13, Cluj-Napoca)

Roots (Bulevardul Eroilor 4, Cluj-Napoca)

Olivo Coffee Roasters (Bulevardul Eroilor 7, Cluj-Napoca)

Klausen Burger (Strada Regele Ferdinand 22, Cluj-Napoca)

Grande Pizza (Strada Teodor Mihali 4, Cluj-Napoca)

Chios Carrousel (Parcul Central, Cluj-Napoca)

Via Restaurant (Strada Inocenţiu Micu Klein 6, Cluj-Napoca)

The Office Canteen (Bulevardul 21 Decembrie, Cluj-Napoca)

Apahida Grill (Calea Dezmirlui 1, Apahida)

2. Pick your Food

[Main] Steak Sandwich - 55.00 RON

[Main] Duck Confit - 78.00 RON

Add to Cart

3. Your Order Summary

Steak Sandwich 55.00 RON

Total: 55.00 RON

Place Order

Driver Dashboard - Online: Alexandru Matei

Available Orders (Pending)

Order #11 | Status: PENDING

Order #13 | Status: PENDING

Order #17 | Status: PENDING

Order #18 | Status: PENDING

Order #30 | Status: PENDING

Order #1033 | Status: PENDING

Order #1036 | Status: PENDING

Order #1037 | Status: PENDING

Order #1038 | Status: PENDING

Order #1039 | Status: PENDING

My Active Deliveries (In Progress)

Order #3 | Status: ARRIVING

Order #32 | Status: ARRIVING

Order #1034 | Status: ARRIVING

Accept Selected

Mark as Delivered

## 6. Further possible improvements:

- Implementing a logout button to make changing between Customer, Driver and Admin easier
- Find a method to hash the passwords and not keep them visible in the database
- Maybe add a functionality where you can contact the driver as a customer